

Connected to base (Python 3.13.9)

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns

plt.rcParams.update({
    'figure.dpi': 130,
    'axes.spines.top': False,
    'axes.spines.right': False,
    'axes.grid': True,
    'grid.alpha': 0.3,
    'axes.titlesize': 13,
    'axes.titleweight': 'bold',
    'axes.labelsize': 11,
    'xtick.labelsize': 9,
    'ytick.labelsize': 9,
    'font.family': 'DejaVu Sans',
})

BLUE = '#378ADD'
GREEN = '#1D9E75'
CORAL = '#D85A30'
PINK = '#D4537E'
PURPLE = '#7F77DD'
AMBER = '#BA7517'
PALETTE = [BLUE, GREEN, CORAL, PINK, PURPLE, AMBER, '#0F6E56', '#639922', '#D4537E']

In [ ]: df = pd.read_csv(r'D:\iec\portfolio2\66634-ev-data\Electric_Vehicle_Population_Data

print(f"Shape      : {df.shape}")
print(f"Columns     : {list(df.columns)}")
df.info()
```

```

Shape      : (177866, 17)
Columns    : ['VIN (1-10)', 'County', 'City', 'State', 'Postal Code', 'Model Year',
'Make', 'Model', 'Electric Vehicle Type', 'Clean Alternative Fuel Vehicle (CAFV) Eli
gibility', 'Electric Range', 'Base MSRP', 'Legislative District', 'DOL Vehicle ID',
'Vehicle Location', 'Electric Utility', '2020 Census Tract']
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 177866 entries, 0 to 177865
Data columns (total 17 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   VIN (1-10)                               177866 non-null  object
1   County                                   177861 non-null  object
2   City                                    177861 non-null  object
3   State                                   177866 non-null  object
4   Postal Code                             177861 non-null  float64
5   Model Year                             177866 non-null  int64
6   Make                                    177866 non-null  object
7   Model                                   177866 non-null  object
8   Electric Vehicle Type                   177866 non-null  object
9   Clean Alternative Fuel Vehicle (CAFV) Eligibility 177866 non-null  object
10  Electric Range                           177866 non-null  int64
11  Base MSRP                               177866 non-null  int64
12  Legislative District                    177477 non-null  float64
13  DOL Vehicle ID                         177866 non-null  int64
14  Vehicle Location                       177857 non-null  object
15  Electric Utility                       177861 non-null  object
16  2020 Census Tract                      177861 non-null  float64
dtypes: float64(3), int64(4), object(10)
memory usage: 23.1+ MB

```

```

In [ ]: null_summary = df.isnull().sum().reset_index()
null_summary.columns = ['Column', 'Null Count']
null_summary['Null %'] = (null_summary['Null Count'] / len(df) * 100).round(2)
null_summary = null_summary[null_summary['Null Count'] > 0].sort_values('Null Count')
print("\nColumns with missing values:")
print(null_summary.to_string(index=False))

```

```

Columns with missing values:
      Column  Null Count  Null %
Legislative District      389    0.22
      Vehicle Location       9    0.01
              County         5    0.00
              City          5    0.00
              Postal Code     5    0.00
      Electric Utility       5    0.00
      2020 Census Tract     5    0.00

```

```

In [ ]: total      = len(df)
bev_count   = (df['Electric Vehicle Type'] == 'Battery Electric Vehicle (BEV)').sum()
phev_count  = (df['Electric Vehicle Type'] == 'Plug-in Hybrid Electric Vehicle (PHEV)').sum()
top_make    = df['Make'].value_counts().idxmax()
top_county  = df['County'].value_counts().idxmax()

print(f"\n{'='*40}")
print(f" Total EVs registered : {total:,}")
print(f" BEV                    : {bev_count:,} ({bev_count/total*100:.1f}%)")

```

```

print(f"   PHEV                : {phev_count:,}  ({phev_count/total*100:.1f}%)"
print(f"   Top manufacturer       : {top_make}")
print(f"   Top county              : {top_county}")
print(f"{'='*40}\n")

```

```

=====
Total EVs registered : 177,866
BEV                  : 139,210  (78.3%)
PHEV                 : 38,656  (21.7%)
Top manufacturer     : TESLA
Top county           : King
=====

```

```

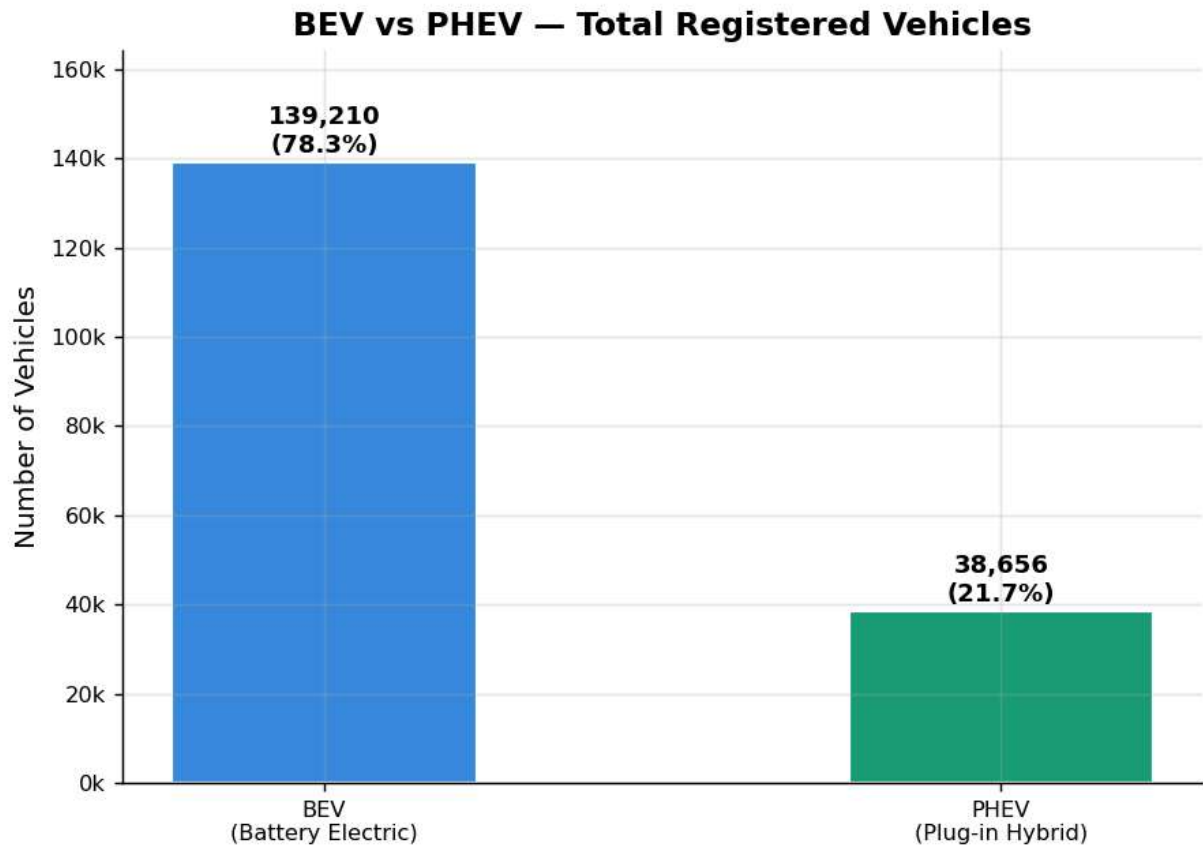
In [ ]: type_counts = df['Electric Vehicle Type'].value_counts()
labels   = ['BEV\n(Battery Electric)', 'PHEV\n(Plug-in Hybrid)']
values   = [
    type_counts.get('Battery Electric Vehicle (BEV)', 0),
    type_counts.get('Plug-in Hybrid Electric Vehicle (PHEV)', 0),
]

fig, ax = plt.subplots(figsize=(7, 5))
bars = ax.bar(labels, values, color=[BLUE, GREEN], width=0.45, edgecolor='white', 1

for bar, val in zip(bars, values):
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 800,
        f'{val:,}\n({val/total*100:.1f}%)',
        ha='center', va='bottom', fontsize=10, fontweight='bold'
    )

ax.set_title('BEV vs PHEV – Total Registered Vehicles')
ax.set_ylabel('Number of Vehicles')
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{int(x/1000)}k'))
ax.set_ylim(0, max(values) * 1.18)
plt.tight_layout()
plt.savefig('bev_vs_phev.png', bbox_inches='tight')
plt.show()

```

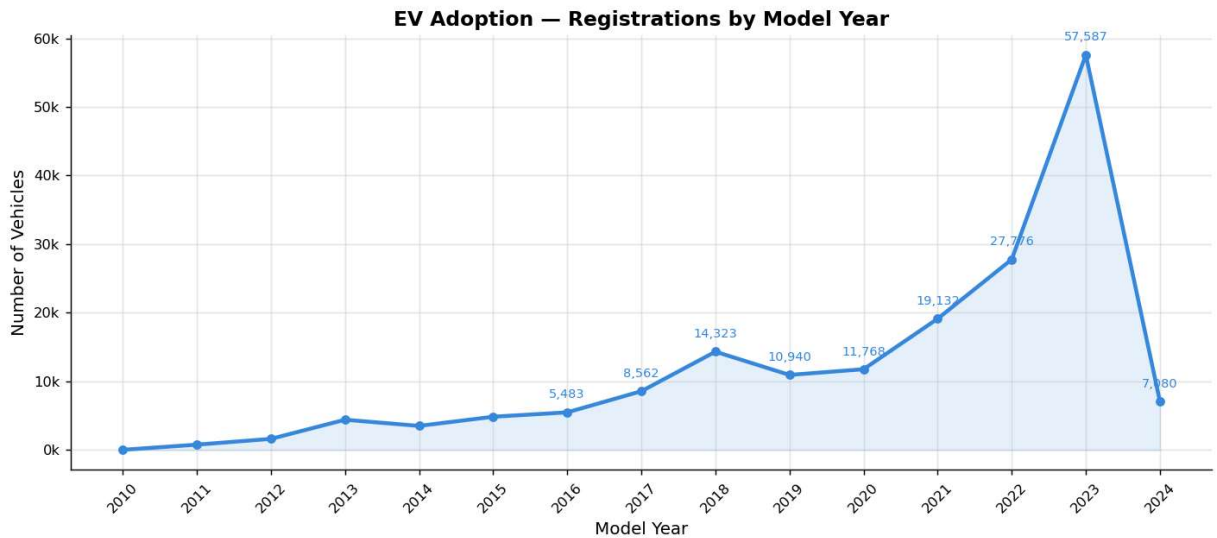


```
In [ ]: adoption = df['Model Year'].value_counts().sort_index()
adoption = adoption[adoption.index >= 2010] # trim noisy early years

fig, ax = plt.subplots(figsize=(11, 5))
ax.fill_between(adoption.index, adoption.values, alpha=0.12, color=BLUE)
ax.plot(adoption.index, adoption.values, color=BLUE, linewidth=2.5, marker='o', mar

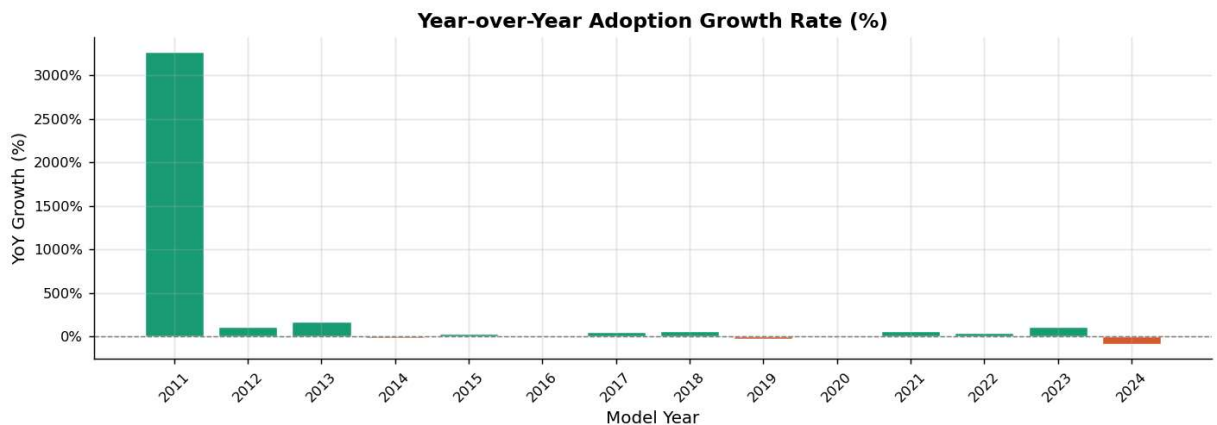
for year, count in adoption.items():
    if count > 5000 or year in [2018, 2022]:
        ax.annotate(
            f'{count:,}',
            (year, count),
            textcoords='offset points', xytext=(0, 9),
            ha='center', fontsize=8, color=BLUE
        )

ax.set_title('EV Adoption — Registrations by Model Year')
ax.set_xlabel('Model Year')
ax.set_ylabel('Number of Vehicles')
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{int(x/1000)}k'))
ax.set_xticks(adoption.index)
ax.tick_params(axis='x', rotation=45)
plt.tight_layout()
plt.savefig('adoption_over_years.png', bbox_inches='tight')
plt.show()
```



```
In [ ]: yoy = adoption.pct_change() * 100
yoy = yoy.dropna()

fig, ax = plt.subplots(figsize=(11, 4))
colors = [GREEN if v >= 0 else CORAL for v in yoy.values]
ax.bar(yoy.index, yoy.values, color=colors, edgecolor='white', linewidth=0.8)
ax.axhline(0, color='gray', linewidth=0.8, linestyle='--')
ax.set_title('Year-over-Year Adoption Growth Rate (%)')
ax.set_xlabel('Model Year')
ax.set_ylabel('YoY Growth (%)')
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{x:.0f}%'))
ax.set_xticks(yoy.index)
ax.tick_params(axis='x', rotation=45)
plt.tight_layout()
plt.savefig('adoption_yoy_growth.png', bbox_inches='tight')
plt.show()
```



```
In [ ]: make_counts = df['Make'].value_counts()
top_n = 9
top_makes = make_counts.head(top_n)
others = make_counts.iloc[top_n:].sum()
make_plot = pd.concat([top_makes, pd.Series({'Others': others})])

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
```

```

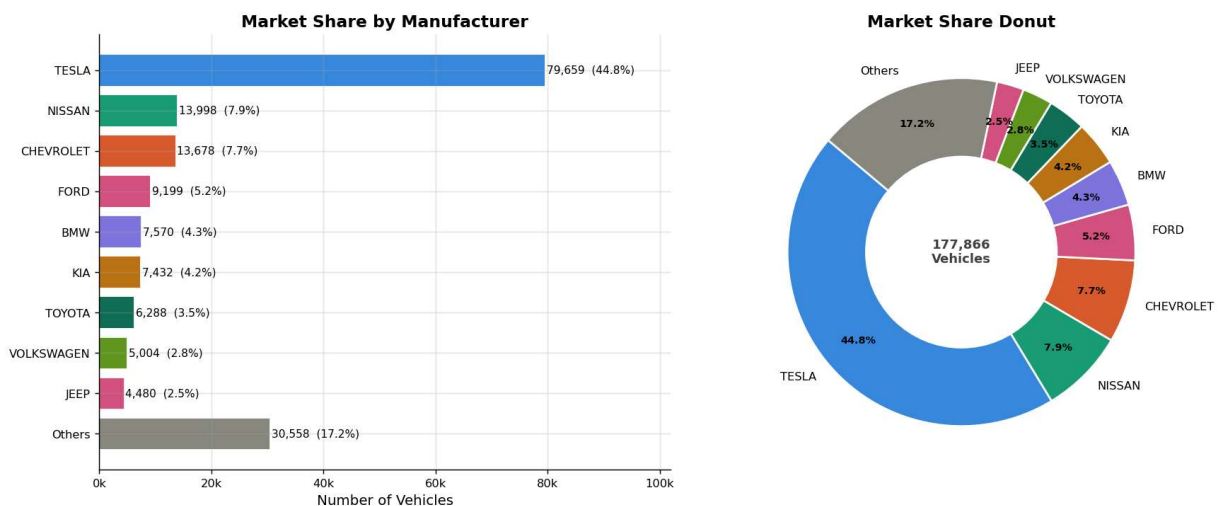
# - Horizontal bar chart
bars = ax1.barh(make_plot.index[::-1], make_plot.values[::-1],
                color=PALETTE[:len(make_plot)][::-1], edgecolor='white')
for bar, val in zip(bars, make_plot.values[::-1]):
    ax1.text(val + 200, bar.get_y() + bar.get_height() / 2,
             f'{val:,} ({val/total*100:.1f})%',
             va='center', fontsize=8.5)
ax1.set_title('Market Share by Manufacturer')
ax1.set_xlabel('Number of Vehicles')
ax1.xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{int(x/1000)}k'))
ax1.set_xlim(0, make_plot.max() * 1.28)

# - Pie / donut chart
wedge_colors = PALETTE[:len(make_plot)]
wedges, texts, autotexts = ax2.pie(
    make_plot.values,
    labels=make_plot.index,
    autopct=lambda p: f'{p:.1f}%' if p > 2.5 else '',
    colors=wedge_colors,
    startangle=140,
    wedgeprops={'edgecolor': 'white', 'linewidth': 1.5},
    pctdistance=0.78
)
for t in autotexts:
    t.set_fontsize(8)
    t.set_fontweight('bold')
centre = plt.Circle((0, 0), 0.55, color='white')
ax2.add_patch(centre)
ax2.text(0, 0, f'{total:,}\nVehicles', ha='center', va='center',
        fontsize=10, fontweight='bold', color='#444')
ax2.set_title('Market Share Donut')

plt.suptitle('EV Manufacturer Market Share - Washington State', fontsize=14, fontwe
plt.tight_layout()
plt.savefig('market_share.png', bbox_inches='tight')
plt.show()

```

EV Manufacturer Market Share — Washington State



```

In [ ]: df['Is Tesla'] = df['Make'] == 'TESLA'
yearly_split = df.groupby(['Model Year', 'Is Tesla']).size().unstack(fill_value=0)

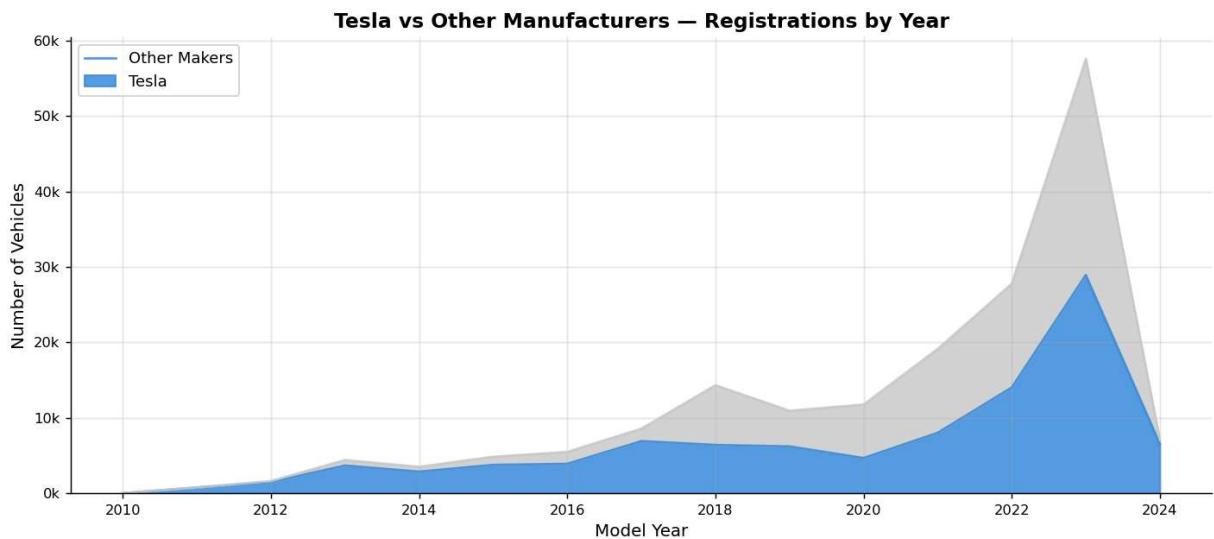
```

```

yearly_split.columns = ['Other Makers', 'Tesla']
yearly_split = yearly_split[yearly_split.index >= 2010]

fig, ax = plt.subplots(figsize=(11, 5))
yearly_split.plot(kind='area', stacked=True, ax=ax,
                  color=[BLUE, '#cccccc'], alpha=0.85)
ax.set_title('Tesla vs Other Manufacturers – Registrations by Year')
ax.set_xlabel('Model Year')
ax.set_ylabel('Number of Vehicles')
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{int(x/1000)}k'))
ax.legend(['Other Makers', 'Tesla'], loc='upper left')
plt.tight_layout()
plt.savefig('tesla_vs_others.png', bbox_inches='tight')
plt.show()

```



```

In [ ]: bev_df = df[df['Electric Vehicle Type'] == 'Battery Electric Vehicle (BEV)'].copy
bev_range = bev_df[bev_df['Electric Range'] > 0]['Electric Range'] # exclude 0-range

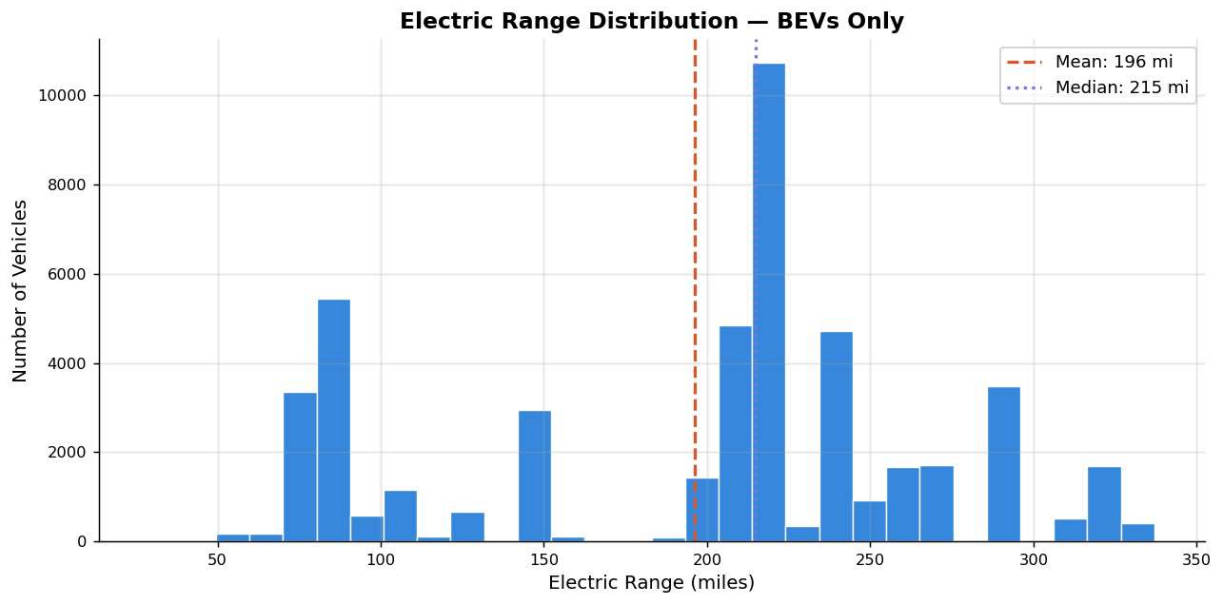
fig, ax = plt.subplots(figsize=(10, 5))
ax.hist(bev_range, bins=30, color=BLUE, edgecolor='white', linewidth=0.8)

mean_r = bev_range.mean()
median_r = bev_range.median()
ax.axvline(mean_r, color=CORAL, linestyle='--', linewidth=1.8, label=f'Mean: {me
ax.axvline(median_r, color=PURPLE, linestyle=':', linewidth=1.8, label=f'Median: {

ax.set_title('Electric Range Distribution – BEVs Only')
ax.set_xlabel('Electric Range (miles)')
ax.set_ylabel('Number of Vehicles')
ax.legend()
plt.tight_layout()
plt.savefig('electric_range_distribution.png', bbox_inches='tight')
plt.show()

print(f"\nBEV Range Stats (excluding 0-range records):")
print(bev_range.describe().apply(lambda x: f'{x:.1f}'))

```



BEV Range Stats (excluding 0-range records):

```
count    47260.0
mean      196.3
std       73.3
min       29.0
25%      149.0
50%      215.0
75%      238.0
max       337.0
```

Name: Electric Range, dtype: object

```
In [ ]: top_bev_makes = (
    bev_df[bev_df['Electric Range'] > 0]
    .groupby('Make')['Electric Range']
    .count()
    .nlargest(6)
    .index.tolist()
)
range_data = bev_df[(bev_df['Make'].isin(top_bev_makes)) & (bev_df['Electric Range']

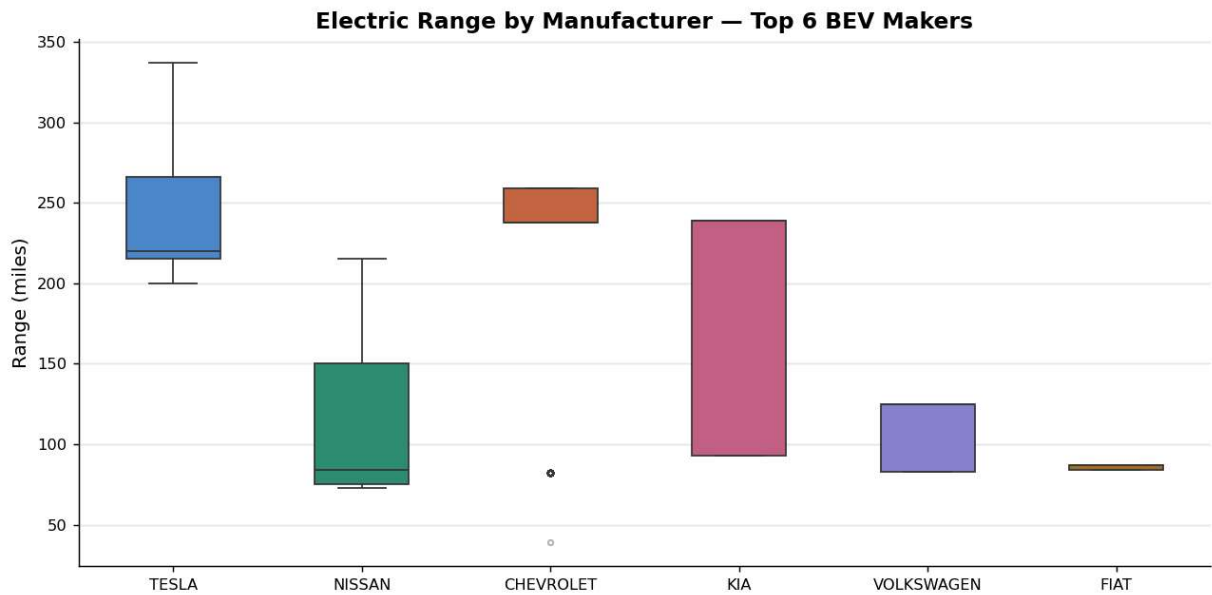
fig, ax = plt.subplots(figsize=(10, 5))
sns.boxplot(
    data=range_data,
    x='Make', y='Electric Range',
    order=top_bev_makes,
    palette=PALETTE[:6],
    width=0.5,
    flierprops={'marker': 'o', 'markersize': 3, 'alpha': 0.4},
    ax=ax
)
ax.set_title('Electric Range by Manufacturer — Top 6 BEV Makers')
ax.set_xlabel('')
ax.set_ylabel('Range (miles)')
plt.tight_layout()
plt.savefig('range_by_make.png', bbox_inches='tight')
plt.show()
```



```
<ipython-input-11-1e860c3c0dbb>:12: FutureWarning:
```

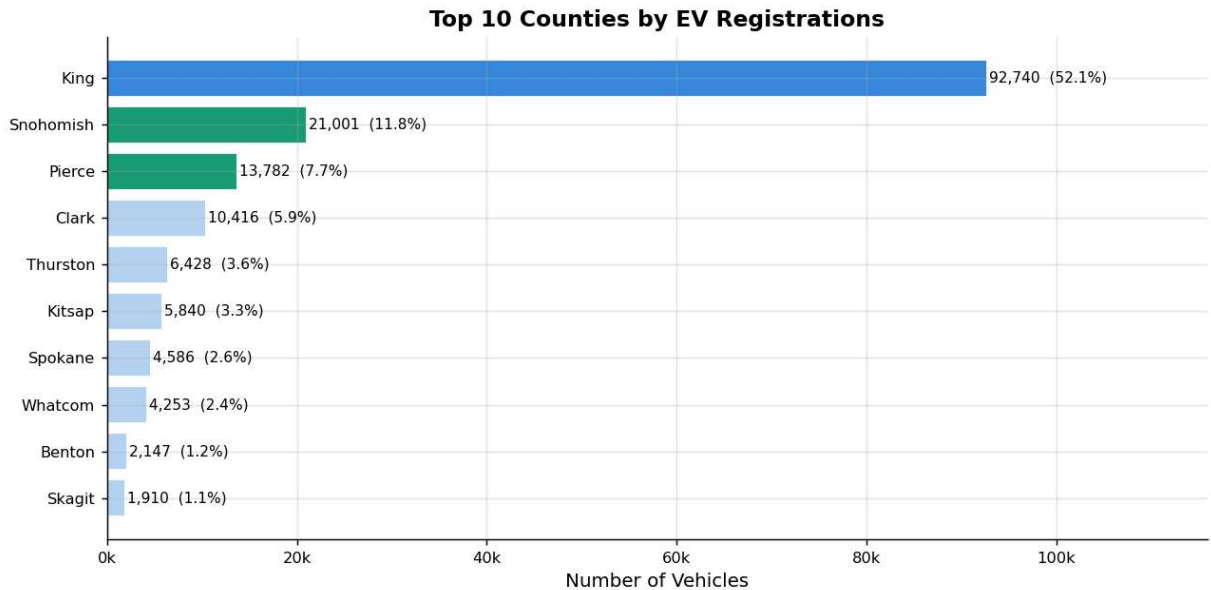
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(
```



```
In [ ]: county_counts = df['County'].value_counts().head(10)

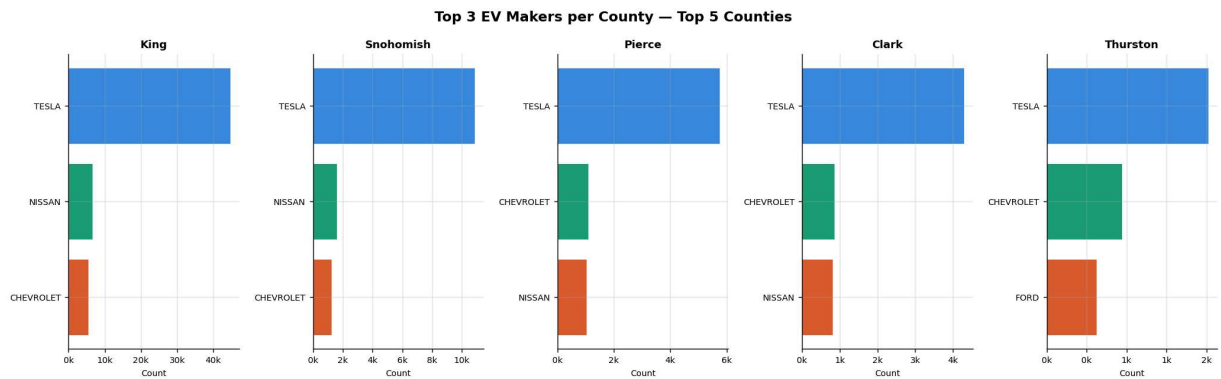
fig, ax = plt.subplots(figsize=(10, 5))
bars = ax.barh(county_counts.index[::-1], county_counts.values[::-1],
               color=[BLUE if i == 0 else GREEN if i < 3 else '#B5D4F4'
                     for i in range(len(county_counts)-1, -1, -1)],
               edgecolor='white')
for bar, val in zip(bars, county_counts.values[::-1]):
    ax.text(val + 200, bar.get_y() + bar.get_height() / 2,
            f'{val:,} ({val/total*100:.1f}%)',
            va='center', fontsize=8.5)
ax.set_title('Top 10 Counties by EV Registrations')
ax.set_xlabel('Number of Vehicles')
ax.xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{int(x/1000)}k'))
ax.set_xlim(0, county_counts.max() * 1.25)
plt.tight_layout()
plt.savefig('top_counties.png', bbox_inches='tight')
plt.show()
```



```
In [ ]: top5_counties = df['County'].value_counts().head(5).index.tolist()
county_make = (
    df[df['County'].isin(top5_counties)]
    .groupby(['County', 'Make'])
    .size()
    .reset_index(name='Count')
)
top3_per_county = (
    county_make
    .sort_values('Count', ascending=False)
    .groupby('County')
    .head(3)
    .reset_index(drop=True)
)

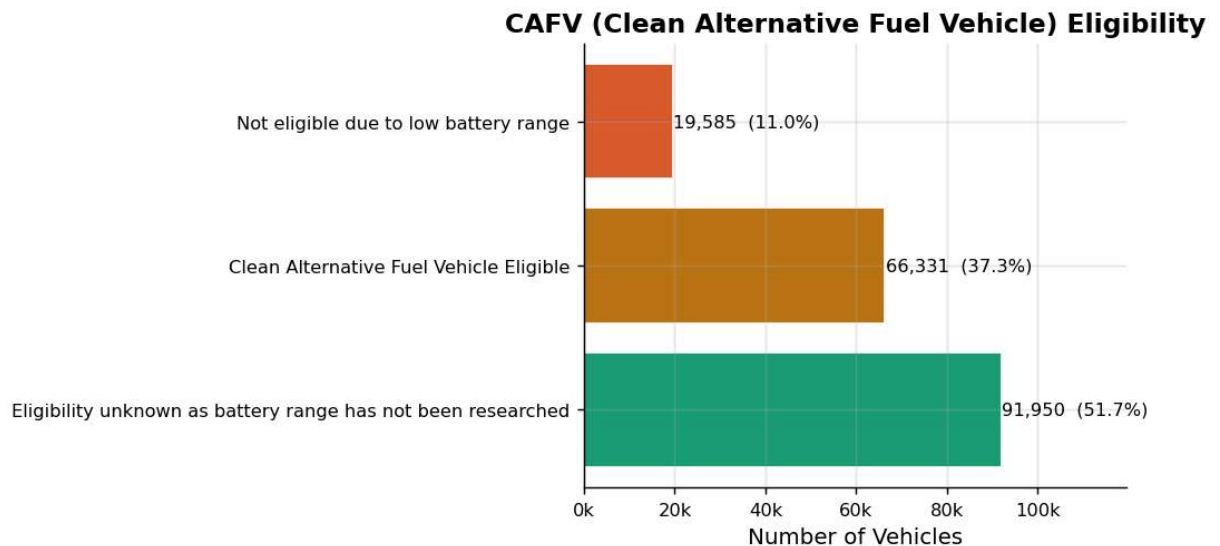
fig, axes = plt.subplots(1, 5, figsize=(16, 5), sharey=False)
for ax, county in zip(axes, top5_counties):
    data = top3_per_county[top3_per_county['County'] == county]
    ax.barh(data['Make'][:-1], data['Count'][:-1],
            color=PALETTE[:3][:-1], edgecolor='white')
    ax.set_title(county, fontsize=10, fontweight='bold')
    ax.set_xlabel('Count', fontsize=8)
    ax.xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{int(x/1000)}'))
    ax.tick_params(labelsize=8)

plt.suptitle('Top 3 EV Makers per County - Top 5 Counties', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig('maker_per_county.png', bbox_inches='tight')
plt.show()
```



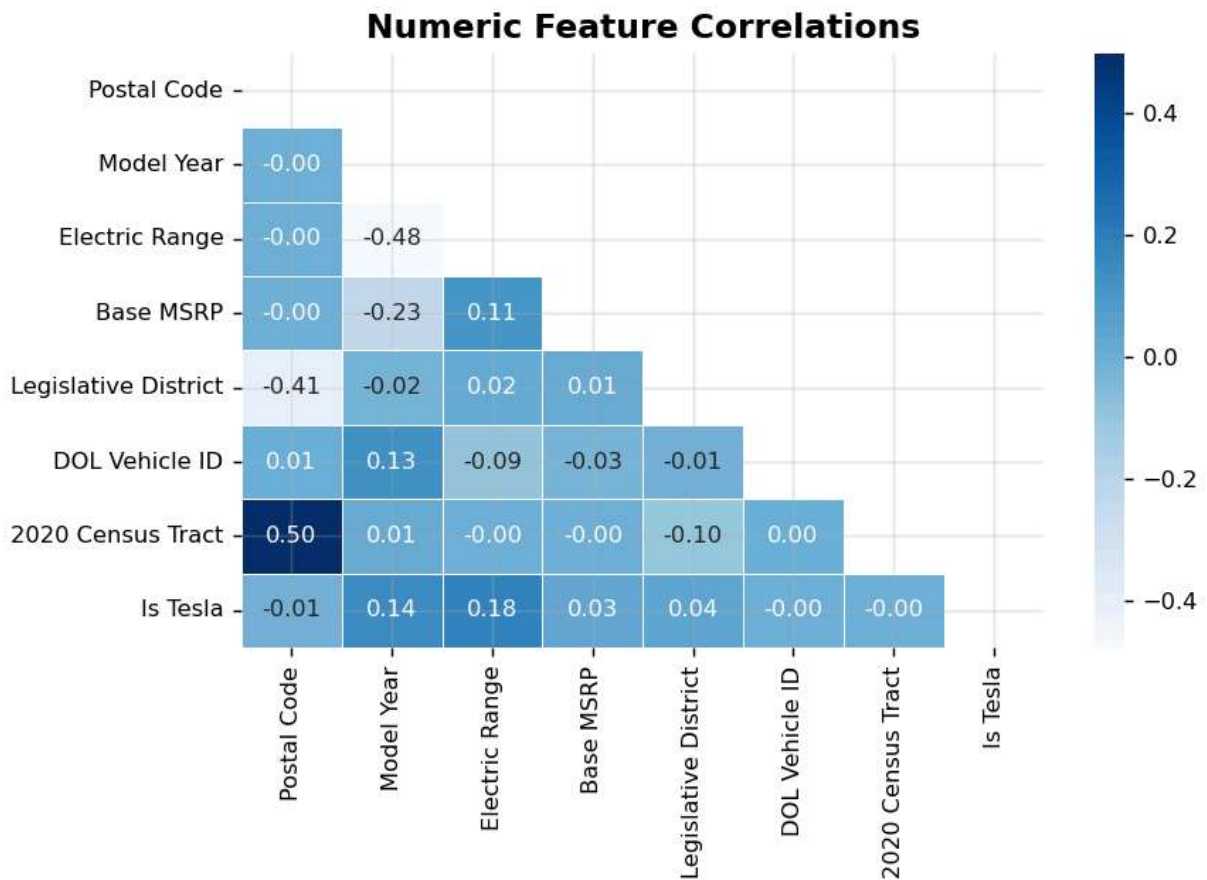
```
In [ ]: cafv_col = 'Clean Alternative Fuel Vehicle (CAFV) Eligibility'
cafv_vals = df[cafv_col].value_counts()

fig, ax = plt.subplots(figsize=(8, 4))
bars = ax.barh(cafv_vals.index, cafv_vals.values,
               color=[GREEN, AMBER, CORAL][:len(cafv_vals)], edgecolor='white')
for bar, val in zip(bars, cafv_vals.values):
    ax.text(val + 100, bar.get_y() + bar.get_height() / 2,
            f'{val:,} ({val/total*100:.1f}%)', va='center', fontsize=9)
ax.set_title('CAFV (Clean Alternative Fuel Vehicle) Eligibility')
ax.set_xlabel('Number of Vehicles')
ax.xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{int(x/1000)}k'))
ax.set_xlim(0, cafv_vals.max() * 1.3)
plt.tight_layout()
plt.savefig('cafv_eligibility.png', bbox_inches='tight')
plt.show()
```



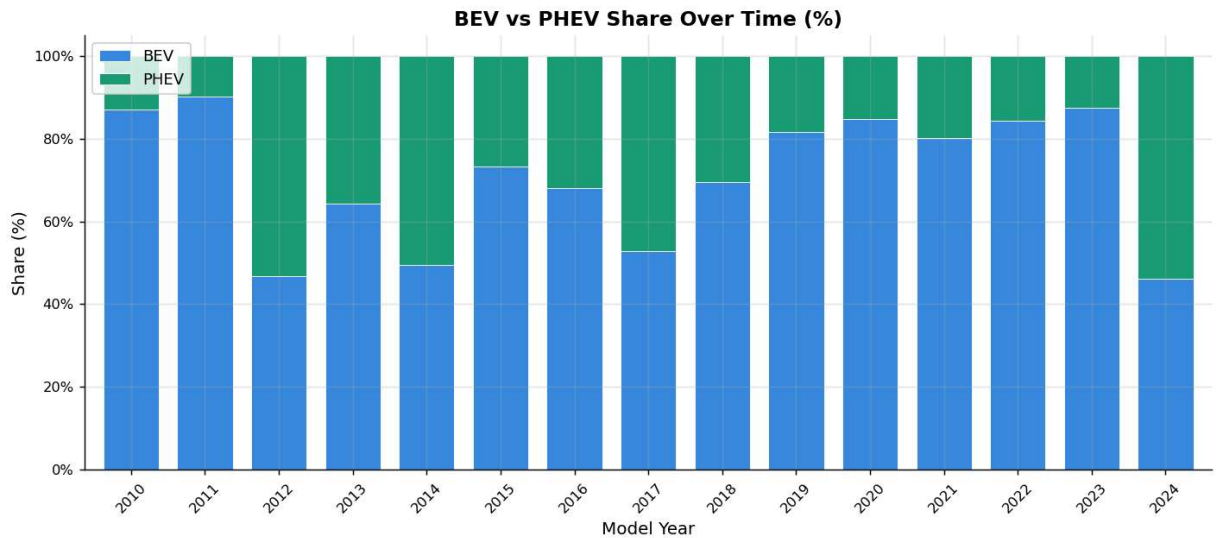
```
In [ ]: fig, ax = plt.subplots(figsize=(7, 5))
corr = df.corr(numeric_only=True)
mask = np.triu(np.ones_like(corr, dtype=bool)) # upper triangle only
sns.heatmap(
    corr, mask=mask, annot=True, fmt='.2f',
    cmap='Blues', linewidths=0.5,
    annot_kws={'size': 9}, ax=ax
)
ax.set_title('Numeric Feature Correlations')
```

```
plt.tight_layout()
plt.savefig('correlation_heatmap.png', bbox_inches='tight')
plt.show()
```



```
In [ ]: yearly_type = (
    df[df['Model Year'] >= 2010]
    .groupby(['Model Year', 'Electric Vehicle Type'])
    .size()
    .unstack(fill_value=0)
)
yearly_type_pct = yearly_type.div(yearly_type.sum(axis=1), axis=0) * 100

fig, ax = plt.subplots(figsize=(11, 5))
yearly_type_pct.plot(kind='bar', stacked=True, ax=ax,
                    color=[BLUE, GREEN], edgecolor='white', linewidth=0.5, width=0.8)
ax.set_title('BEV vs PHEV Share Over Time (%)')
ax.set_xlabel('Model Year')
ax.set_ylabel('Share (%)')
ax.set_ylim(0, 105)
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{x:.0f}%'))
ax.legend(['BEV', 'PHEV'], loc='upper left')
ax.tick_params(axis='x', rotation=45)
plt.tight_layout()
plt.savefig('bev_phev_share_over_time.png', bbox_inches='tight')
plt.show()
```



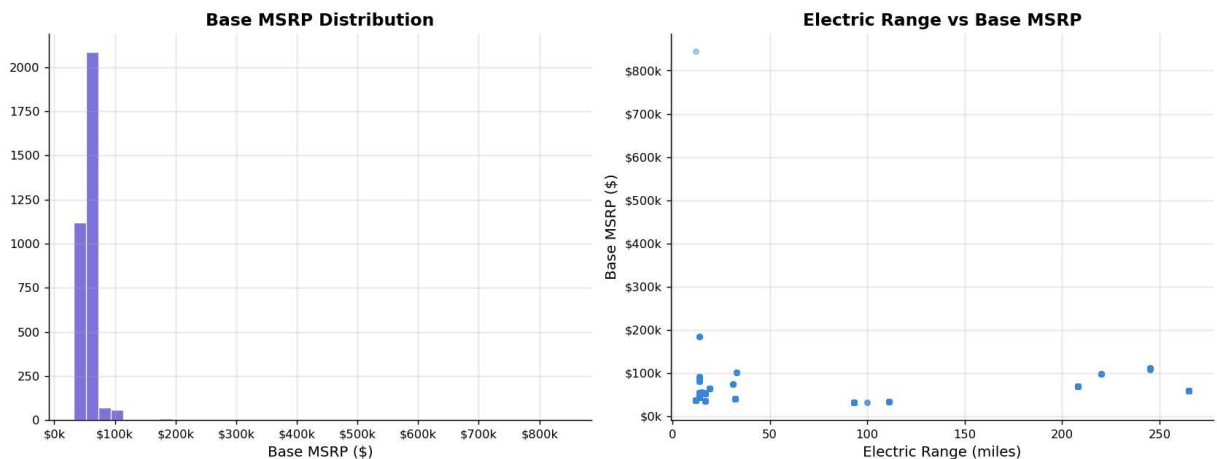
```
In [ ]: msrp_df = df[df['Base MSRP'] > 0].copy()

if len(msrp_df) > 100:
    fig, axes = plt.subplots(1, 2, figsize=(13, 5))

    # Distribution
    axes[0].hist(msrp_df['Base MSRP'], bins=40, color='PURPLE', edgecolor='white', li
    axes[0].set_title('Base MSRP Distribution')
    axes[0].set_xlabel('Base MSRP ($)')
    axes[0].xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'${int(x/

    # MSRP vs Range
    axes[1].scatter(msrp_df['Electric Range'], msrp_df['Base MSRP'],
                    alpha=0.4, color=BLUE, s=15)
    axes[1].set_title('Electric Range vs Base MSRP')
    axes[1].set_xlabel('Electric Range (miles)')
    axes[1].set_ylabel('Base MSRP ($)')
    axes[1].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'${int(x/

    plt.tight_layout()
    plt.savefig('msrp_analysis.png', bbox_inches='tight')
    plt.show()
else:
    print("Not enough non-zero MSRP records to plot (common in this dataset - most
```



```

In [ ]: # — recompute all needed variables so this cell runs standalone —
total      = len(df)
bev_count  = (df['Electric Vehicle Type'] == 'Battery Electric Vehicle (BEV)').sum()
phev_count = (df['Electric Vehicle Type'] == 'Plug-in Hybrid Electric Vehicle (PHEV)').sum()

adoption    = df['Model Year'].value_counts().sort_index()
adoption    = adoption[adoption.index >= 2010]

bev_df      = df[df['Electric Vehicle Type'] == 'Battery Electric Vehicle (BEV)'].copy()
bev_range   = bev_df[bev_df['Electric Range'] > 0]['Electric Range']

yearly_type = (
    df[df['Model Year'] >= 2010]
    .groupby(['Model Year', 'Electric Vehicle Type'])
    .size()
    .unstack(fill_value=0)
)
yearly_type_pct = yearly_type.div(yearly_type.sum(axis=1), axis=0) * 100
# —————

fig, axes = plt.subplots(2, 3, figsize=(16, 10))
fig.suptitle('Washington State EV Population - Full Dashboard', fontsize=15, fontweight='bold')

# Panel 1: BEV vs PHEV
ax = axes[0, 0]
ax.bar(['BEV', 'PHEV'], [bev_count, phev_count], color=[BLUE, GREEN], edgecolor='white')
ax.set_title('BEV vs PHEV')
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{int(x/1000)}k'))

# Panel 2: Adoption over time
ax = axes[0, 1]
ax.fill_between(adoption.index, adoption.values, alpha=0.15, color=BLUE)
ax.plot(adoption.index, adoption.values, color=BLUE, linewidth=2, marker='o', markersize=10)
ax.set_title('Adoption by Model Year')
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{int(x/1000)}k'))
ax.tick_params(axis='x', rotation=45)

# Panel 3: Top 8 makes
ax = axes[0, 2]
top8 = df['Make'].value_counts().head(8)
ax.barh(top8.index[::-1], top8.values[::-1], color=PALETTE[:8][::-1], edgecolor='white')
ax.set_title('Top 8 Manufacturers')
ax.xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{int(x/1000)}k'))

# Panel 4: Electric range histogram (BEV > 0)
ax = axes[1, 0]
ax.hist(bev_range, bins=25, color=BLUE, edgecolor='white', linewidth=0.7)
ax.axvline(bev_range.mean(), color=CORAL, linestyle='--', linewidth=1.5, label=f'Mean BEV Range')
ax.set_title('BEV Electric Range')
ax.set_xlabel('Miles')
ax.legend(fontsize=8)

# Panel 5: Top 8 counties
ax = axes[1, 1]
top8_county = df['County'].value_counts().head(8)

```

```

ax.barh(top8_county.index[::-1], top8_county.values[::-1],
        color=[BLUE if i == 7 else '#B5D4F4' for i in range(8)], edgecolor='white')
ax.set_title('Top 8 Counties')
ax.xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{int(x/1000)}k'))

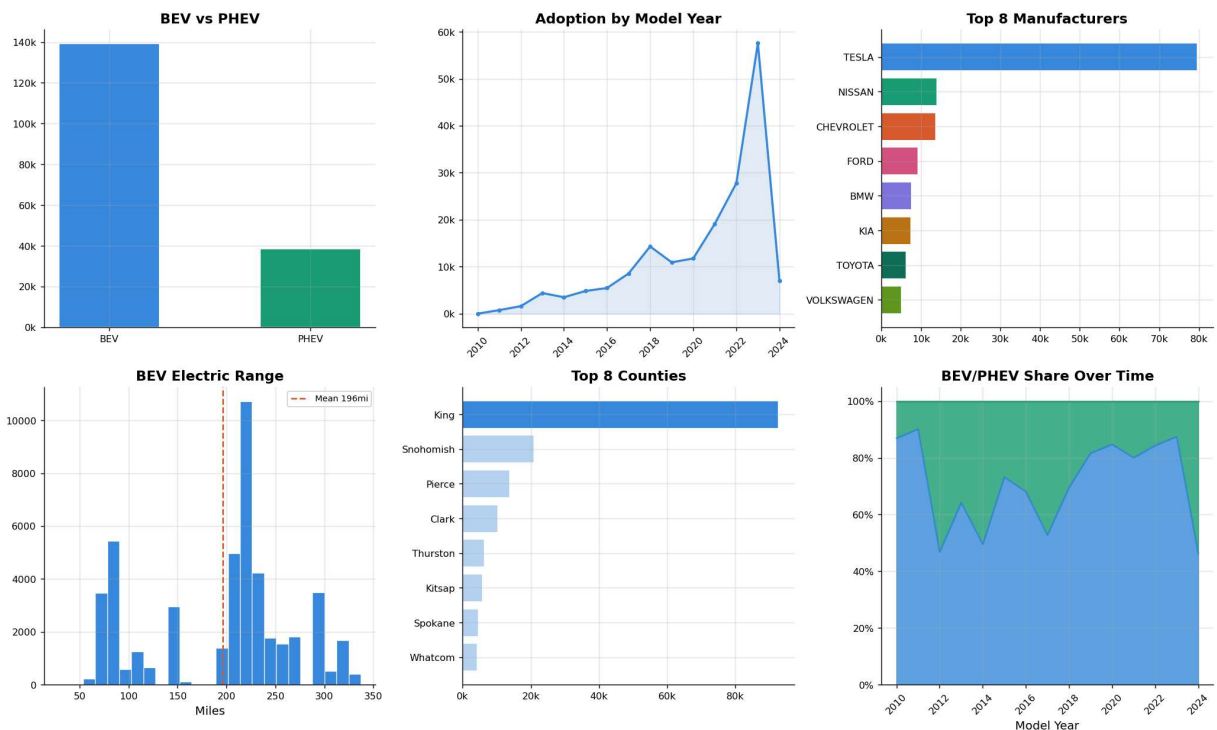
# Panel 6: BEV vs PHEV share over time
ax = axes[1, 2]
yearly_type_pct.plot(kind='area', stacked=True, ax=ax,
                     color=[BLUE, GREEN], alpha=0.8, legend=False)
ax.set_title('BEV/PHEV Share Over Time')
ax.set_xlabel('Model Year')
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{x:.0f}%'))
ax.tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.savefig('full_dashboard.png', bbox_inches='tight', dpi=150)
plt.show()

print("\nAll charts saved as PNG files in the working directory.")
print("Done!")

```

Washington State EV Population — Full Dashboard



All charts saved as PNG files in the working directory.
Done!

```

In [ ]: # — recompute all needed variables so this cell runs standalone —
total      = len(df)
bev_count  = (df['Electric Vehicle Type'] == 'Battery Electric Vehicle (BEV)').sum()
phev_count = (df['Electric Vehicle Type'] == 'Plug-in Hybrid Electric Vehicle (PHEV)').sum()

adoption    = df['Model Year'].value_counts().sort_index()
adoption    = adoption[adoption.index >= 2010]

bev_df      = df[df['Electric Vehicle Type'] == 'Battery Electric Vehicle (BEV)'].co

```

```

bev_range = bev_df[bev_df['Electric Range'] > 0]['Electric Range']

yearly_type = (
    df[df['Model Year'] >= 2010]
    .groupby(['Model Year', 'Electric Vehicle Type'])
    .size()
    .unstack(fill_value=0)
)
yearly_type_pct = yearly_type.div(yearly_type.sum(axis=1), axis=0) * 100
# -----

fig, axes = plt.subplots(2, 3, figsize=(16, 10))
fig.suptitle('Washington State EV Population - Full Dashboard', fontsize=15, fontwe

# Panel 1: BEV vs PHEV
ax = axes[0, 0]
ax.bar(['BEV', 'PHEV'], [bev_count, phev_count], color=[BLUE, GREEN], edgecolor='wh
ax.set_title('BEV vs PHEV')
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{int(x/1000)}k'))

# Panel 2: Adoption over time
ax = axes[0, 1]
ax.fill_between(adoption.index, adoption.values, alpha=0.15, color=BLUE)
ax.plot(adoption.index, adoption.values, color=BLUE, linewidth=2, marker='o', marke
ax.set_title('Adoption by Model Year')
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{int(x/1000)}k'))
ax.tick_params(axis='x', rotation=45)

# Panel 3: Top 8 makes
ax = axes[0, 2]
top8 = df['Make'].value_counts().head(8)
ax.barh(top8.index[::-1], top8.values[::-1], color=PALETTE[:8][::-1], edgecolor='wh
ax.set_title('Top 8 Manufacturers')
ax.xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{int(x/1000)}k'))

# Panel 4: Electric range histogram (BEV > 0)
ax = axes[1, 0]
ax.hist(bev_range, bins=25, color=BLUE, edgecolor='white', linewidth=0.7)
ax.axvline(bev_range.mean(), color=CORAL, linestyle='--', linewidth=1.5, label=f'Me
ax.set_title('BEV Electric Range')
ax.set_xlabel('Miles')
ax.legend(fontsize=8)

# Panel 5: Top 8 counties
ax = axes[1, 1]
top8_county = df['County'].value_counts().head(8)
ax.barh(top8_county.index[::-1], top8_county.values[::-1],
        color=[BLUE if i == 7 else '#B5D4F4' for i in range(8)], edgecolor='white')
ax.set_title('Top 8 Counties')
ax.xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{int(x/1000)}k'))

# Panel 6: BEV vs PHEV share over time
ax = axes[1, 2]
yearly_type_pct.plot(kind='area', stacked=True, ax=ax,
                    color=[BLUE, GREEN], alpha=0.8, legend=False)
ax.set_title('BEV/PHEV Share Over Time')

```



```

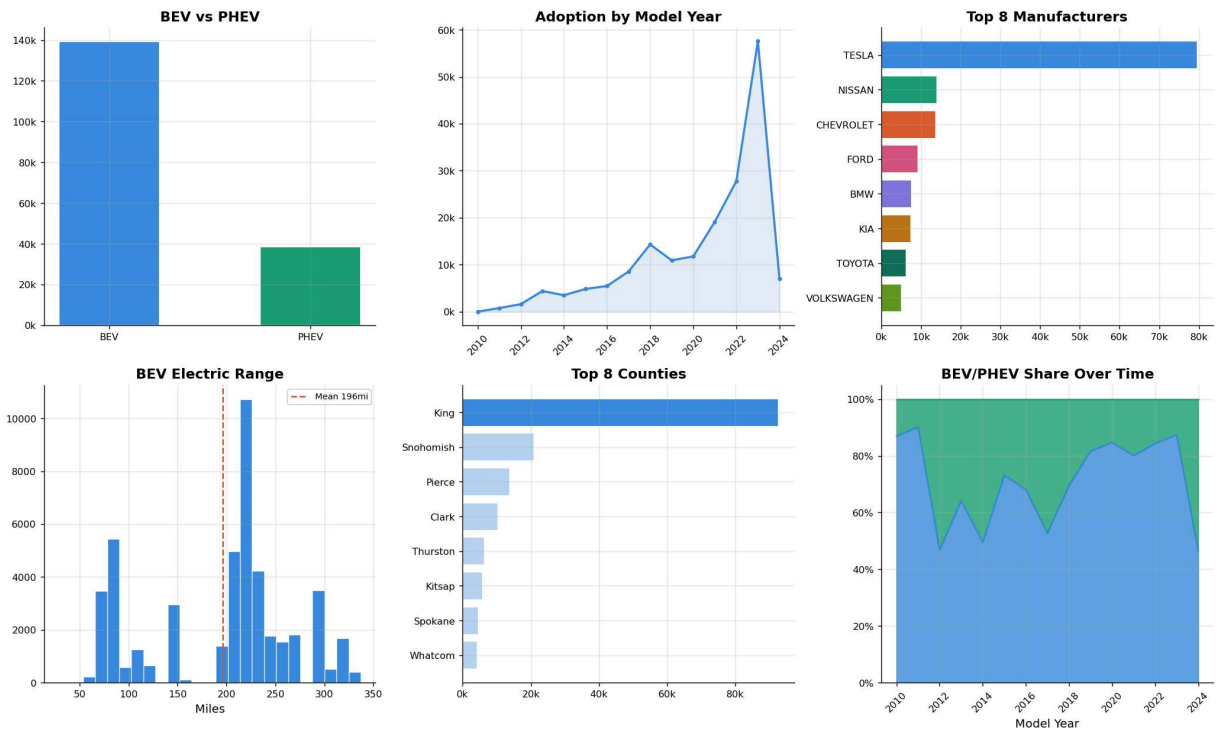
ax.set_xlabel('Model Year')
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{x:.0f}%'))
ax.tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.savefig('full_dashboard.png', bbox_inches='tight', dpi=150)
plt.show()

print("\nAll charts saved as PNG files in the working directory.")
print("Done!")

```

Washington State EV Population — Full Dashboard



All charts saved as PNG files in the working directory.
Done!